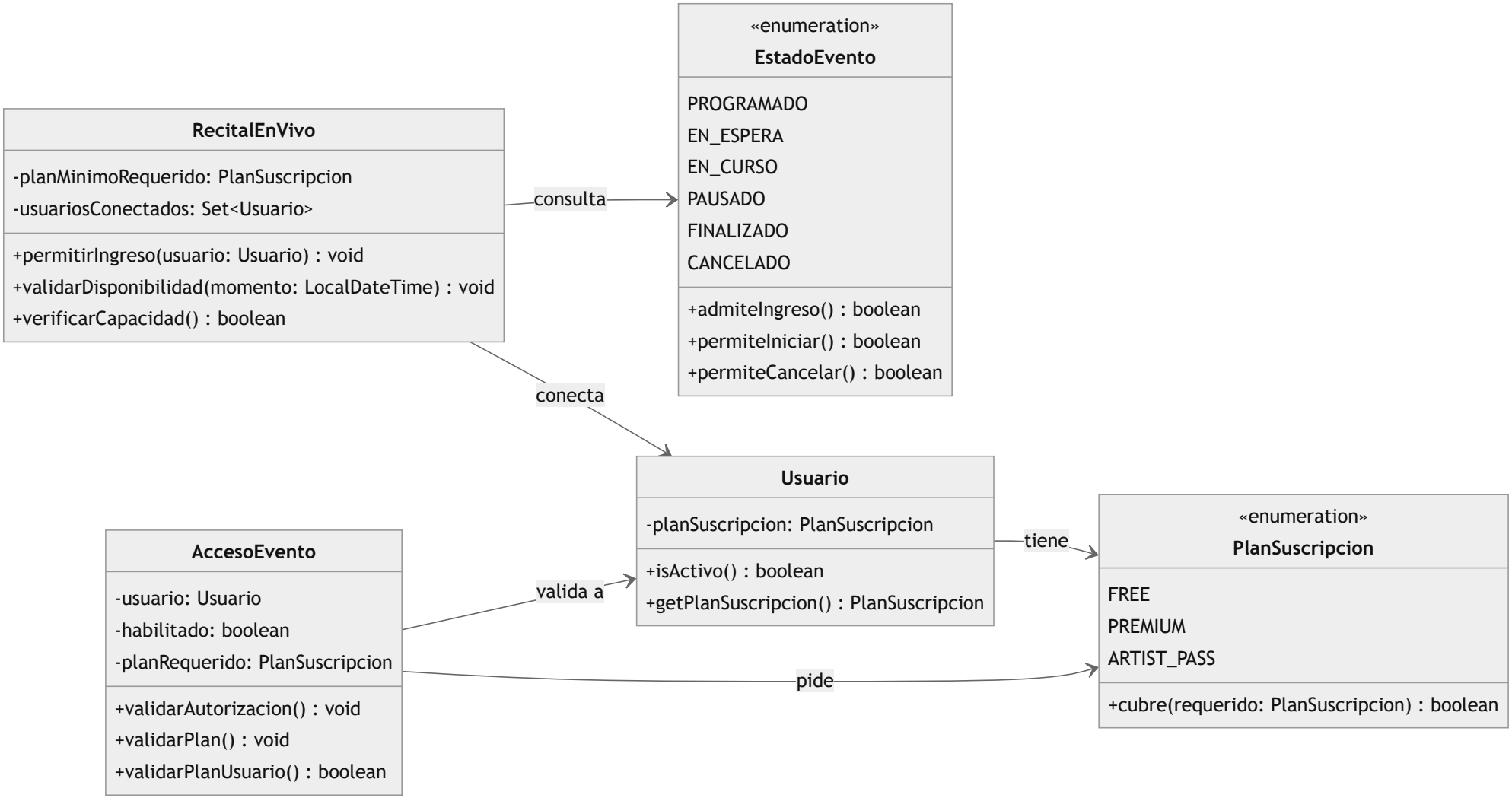
Diagramas GRASP

1. Experto en información

¿Quién decide si un usuario puede entrar a un recital?

Ninguna clase tiene toda la información, así que la decisión se reparte entre las clases que tienen cada dato:

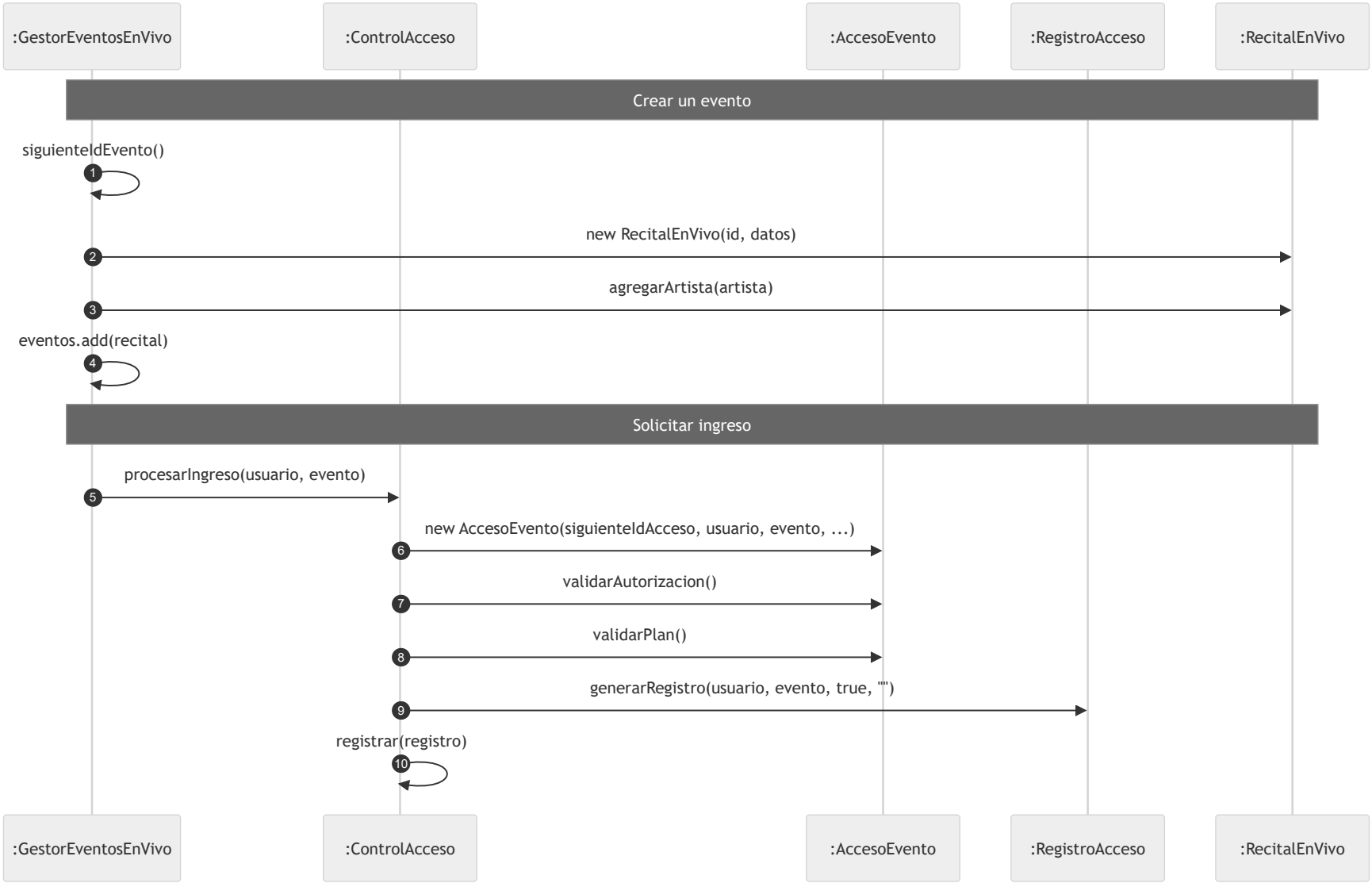
- `AccesoEvento` conoce al usuario, si está habilitado y qué plan pide el evento. Por eso valida la autorización y el plan.
- `PlanSuscripcion` sabe si un plan cubre a otro (`cubre`).
- `RecitalEnVivo` conoce su estado, su horario, su capacidad y quiénes están conectados. Por eso es el que permite o rechaza el ingreso.
- `EstadoEvento` sabe en qué estados se puede entrar y qué cambios de estado están permitidos.


«enumeration»
EstadoEvento
PROGRAMADO
EN_ESPERA
EN_CURSO
PAUSADO
FINALIZADO
CANCELADO
+admiteIngreso() : boolean
+permiteIniciar() : boolean
+permiteCancelar() : boolean
RecitalEnVivo
-planMinimoRequerido: PlanSuscripcion
-usuariosConectados: Set<Usuario>
+permitirIngreso(usuario: Usuario) : void
+validarDisponibilidad(momento: LocalDateTime) : void
+verificarCapacidad() : boolean
consulta
conecta
Usuario
-planSuscripcion: PlanSuscripcion
+isActivo() : boolean
+getPlanSuscripcion() : PlanSuscripcion
tiene
«enumeration»
PlanSuscripcion
FREE
PREMIUM
ARTIST_PASS
+cubre(requerido: PlanSuscripcion) : boolean
AccesoEvento
-usuario: Usuario
-habilitado: boolean
-planRequerido: PlanSuscripcion
+validarAutorizacion() : void
+validarPlan() : void
+validarPlanUsuario() : boolean
valida a
pide

2. Creador

¿Quién crea cada objeto?

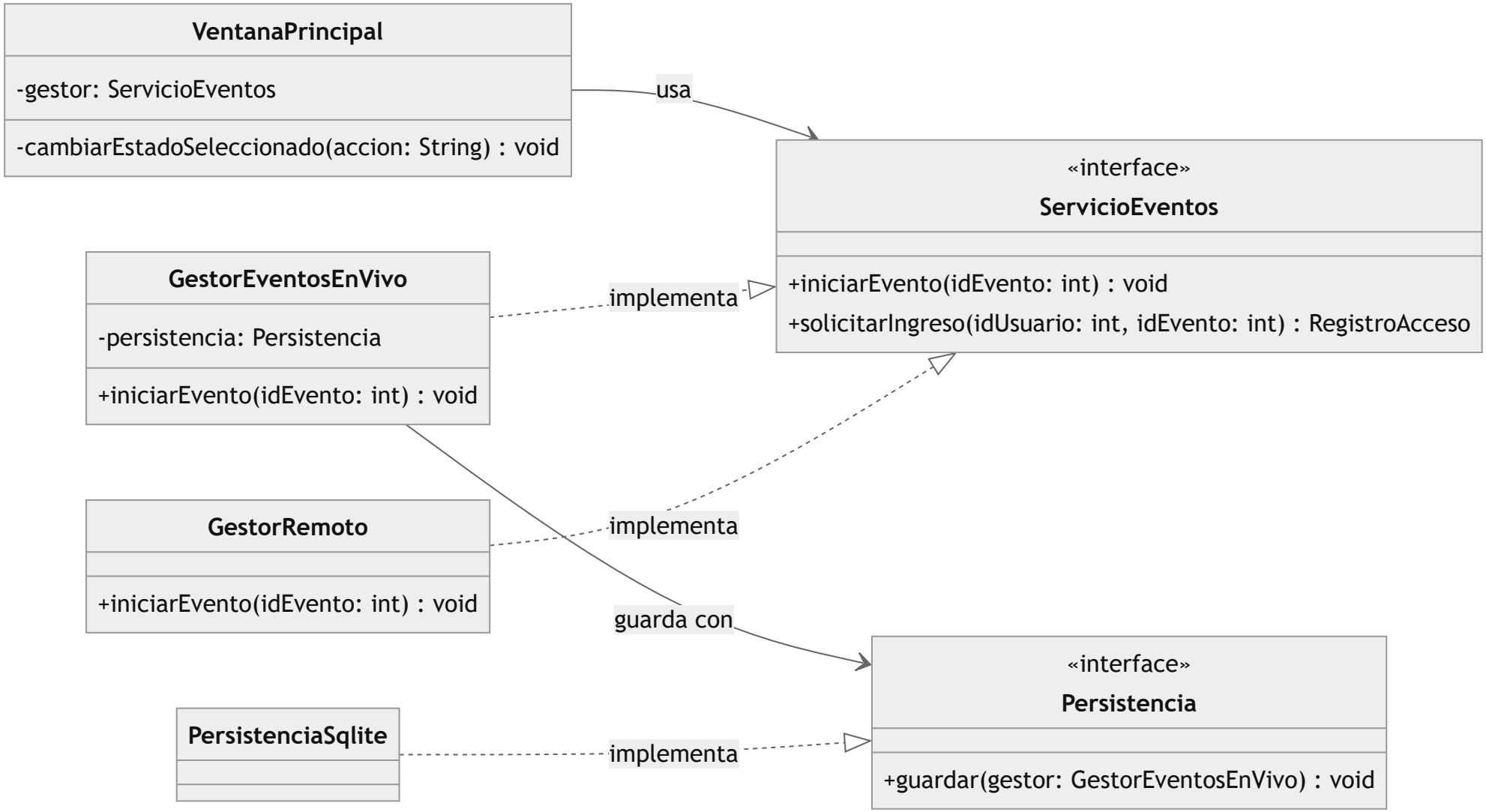
- `ControlAcceso` **crea** `AccesoEvento`. Tiene los datos para armarlo (el usuario, el evento y el número de acceso siguiente) y es el que lo usa enseguida para validar.
- `ControlAcceso` **crea los** `RegistroAcceso` por medio de `generarRegistro` y `registrarIntentoFallido`, porque es la clase que guarda la lista de registros.
- `GestorEventosEnVivo` **crea** `RecitalEnVivo`, porque contiene la lista de eventos y es quien sabe cuál es el próximo ID libre.


:GestorEventosEnVivo
:ControlAcceso
:AccesoEvento
:RegistroAcceso
:RecitalEnVivo
Crear un evento
siguienteIdEvento()
1
new RecitalEnVivo(id, datos)
2
agregarArtista(artista)
3
eventos.add(recital)
4
Solicitar ingreso
procesarIngreso(usuario, evento)
5
new AccesoEvento(siguienteIdAcceso, usuario, evento, ...)
6
validarAutorizacion()
7
validarPlan()
8
generarRegistro(usuario, evento, true, "")
9
registrar(registro)
10
:GestorEventosEnVivo
:ControlAcceso
:AccesoEvento
:RegistroAcceso
:RecitalEnVivo

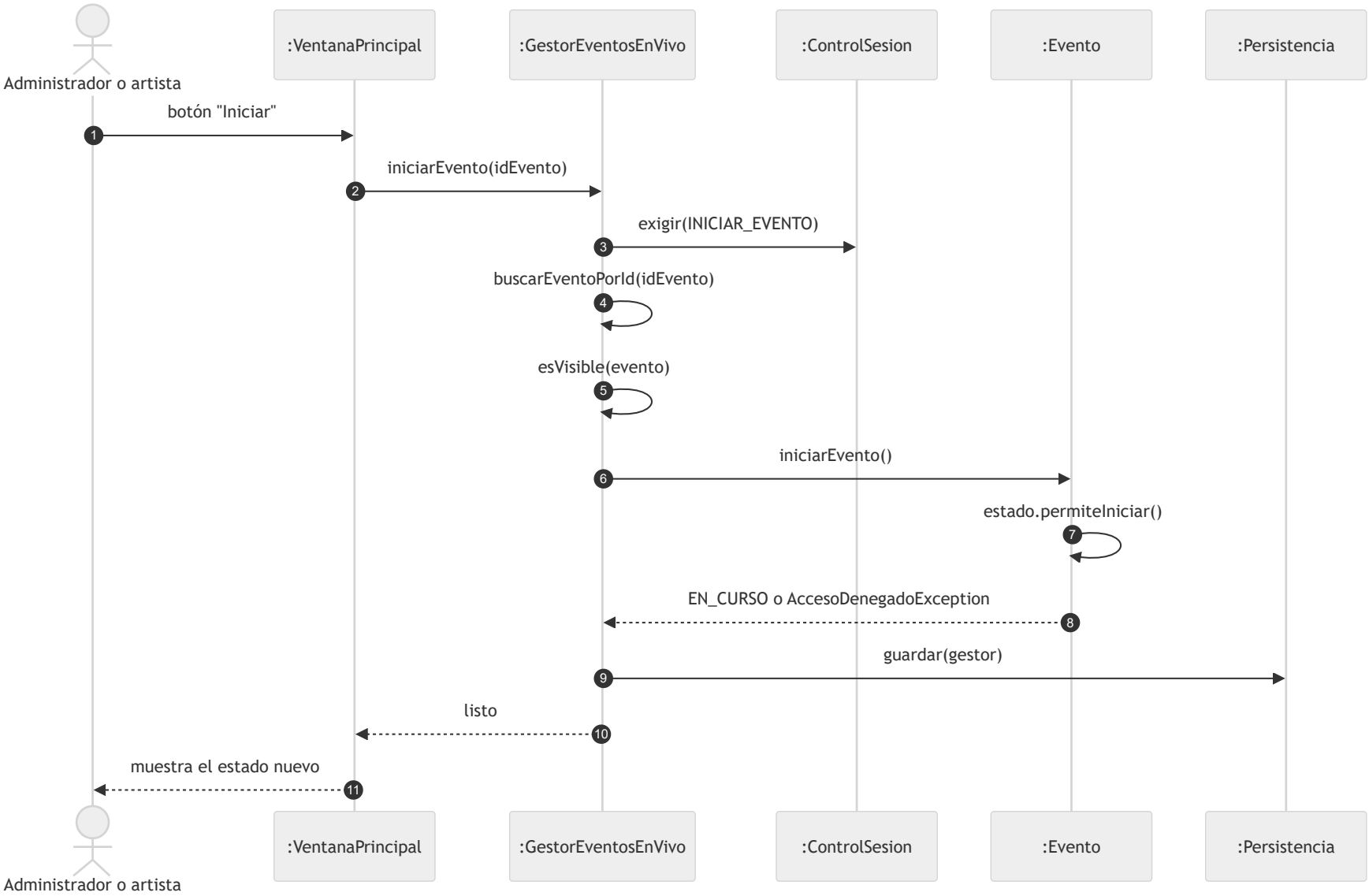
3. Bajo acoplamiento

¿Cómo hacemos para que la ventana no dependa de las reglas del evento ni de dónde se guardan los datos?

La ventana (`VentanaPrincipal`) y la consola (`MenuAcciones`) solo conocen la interfaz `ServicioEventos`. No llaman a métodos de `Evento`, no saben qué estados son válidos y no saben si los datos están en este equipo o en un servidor. La interfaz la implementan `GestorEventosEnVivo` (local) y `GestorRemoto` (que manda el pedido al servidor), y la vista funciona igual con cualquiera de los dos. Lo mismo pasa con el guardado: el gestor depende de la interfaz `Persistencia` y no de SQLite.



Así queda, por ejemplo, el inicio de un evento:

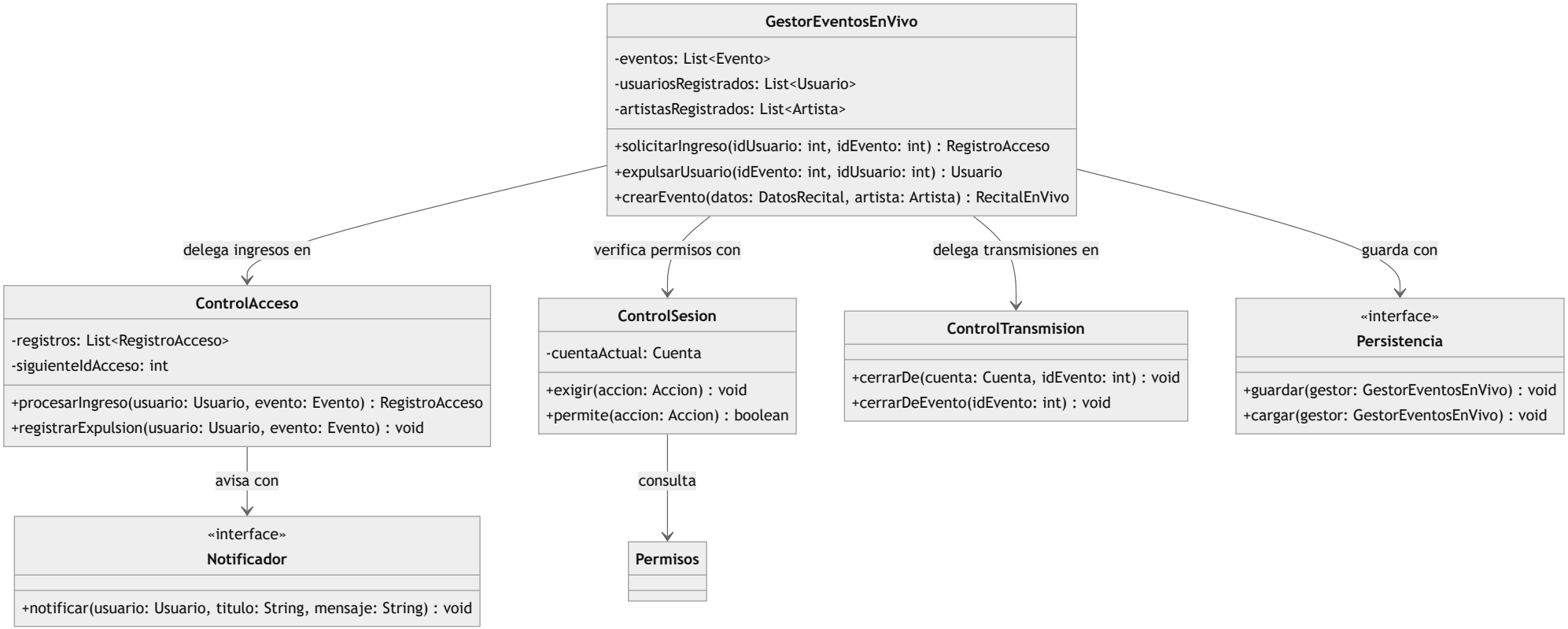

Administrador o artista
:VentanaPrincipal
:GestorEventosEnVivo
:ControlSesion
:Evento
:Persistencia
botón "Iniciar"
1
iniciarEvento(idEvento)
2
exigir(INICIAR_EVENTO)
3
buscarEventoPorId(idEvento)
4
esVisible(evento)
5
iniciarEvento()
6
estado.permiteIniciar()
7
EN_CURSO o AccesoDenegadoException
8
guardar(gestor)
9
listo
10
muestra el estado nuevo
11
Administrador o artista
:VentanaPrincipal
:GestorEventosEnVivo
:ControlSesion
:Evento
:Persistencia

4. Alta cohesión

¿Cómo evitamos que el gestor haga todo?

En la versión 0 el gestor resolvía los ingresos, guardaba en archivos, controlaba la sesión y armaba los avisos. Ahora coordina y le pasa cada tema a una clase que se ocupa solo de eso:

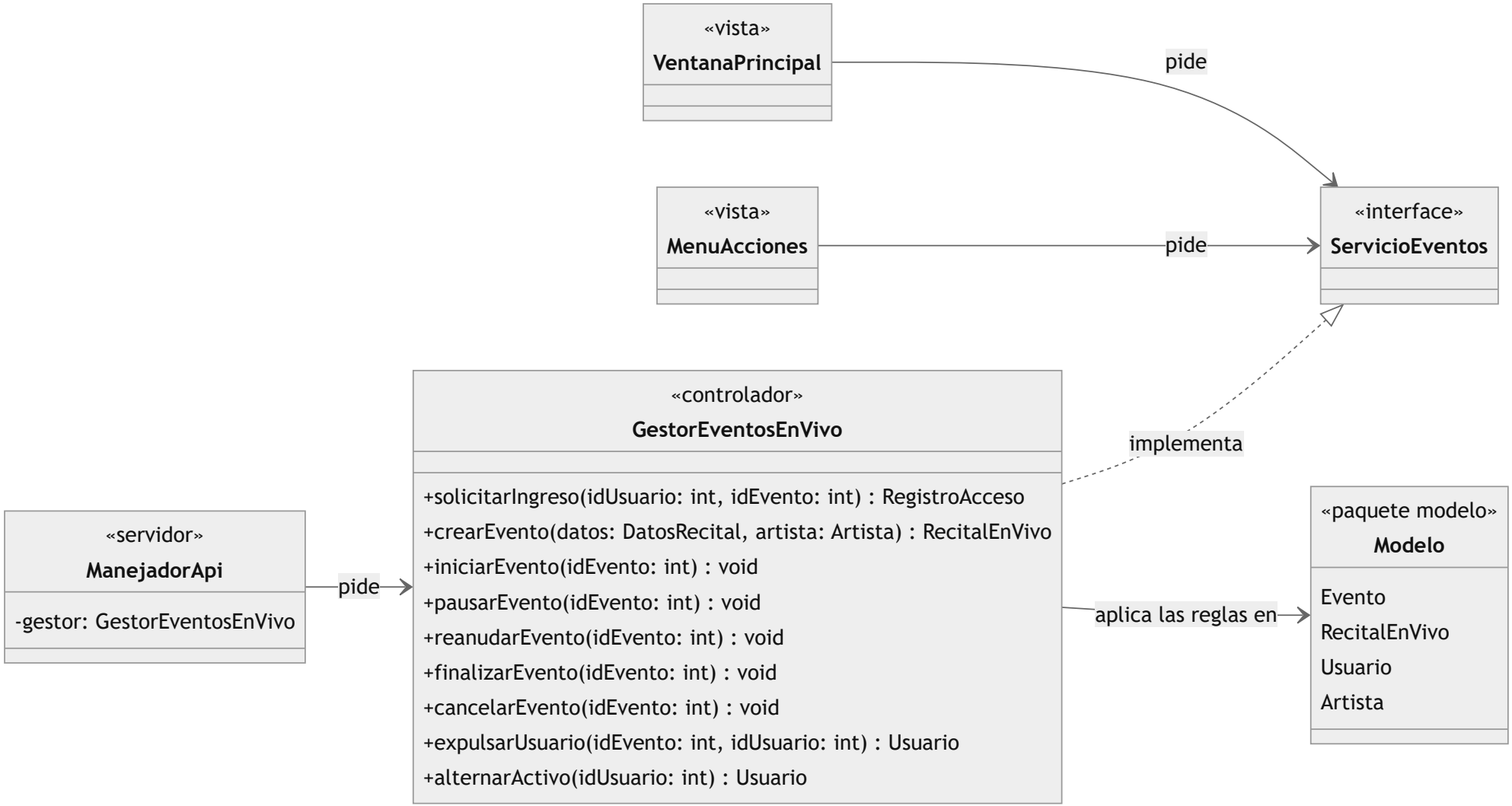
- `ControlAcceso`: decide los ingresos y guarda la traza de cada intento.
- `ControlSesion`: sabe quién inició sesión y, con `Permisos`, qué puede hacer cada rol.
- `ControlTransmision`: maneja las transmisiones y las cierra cuando hace falta.
- `Persistencia`: guarda y carga los datos.
- `Notificador`: avisa a los usuarios.



5. Controlador

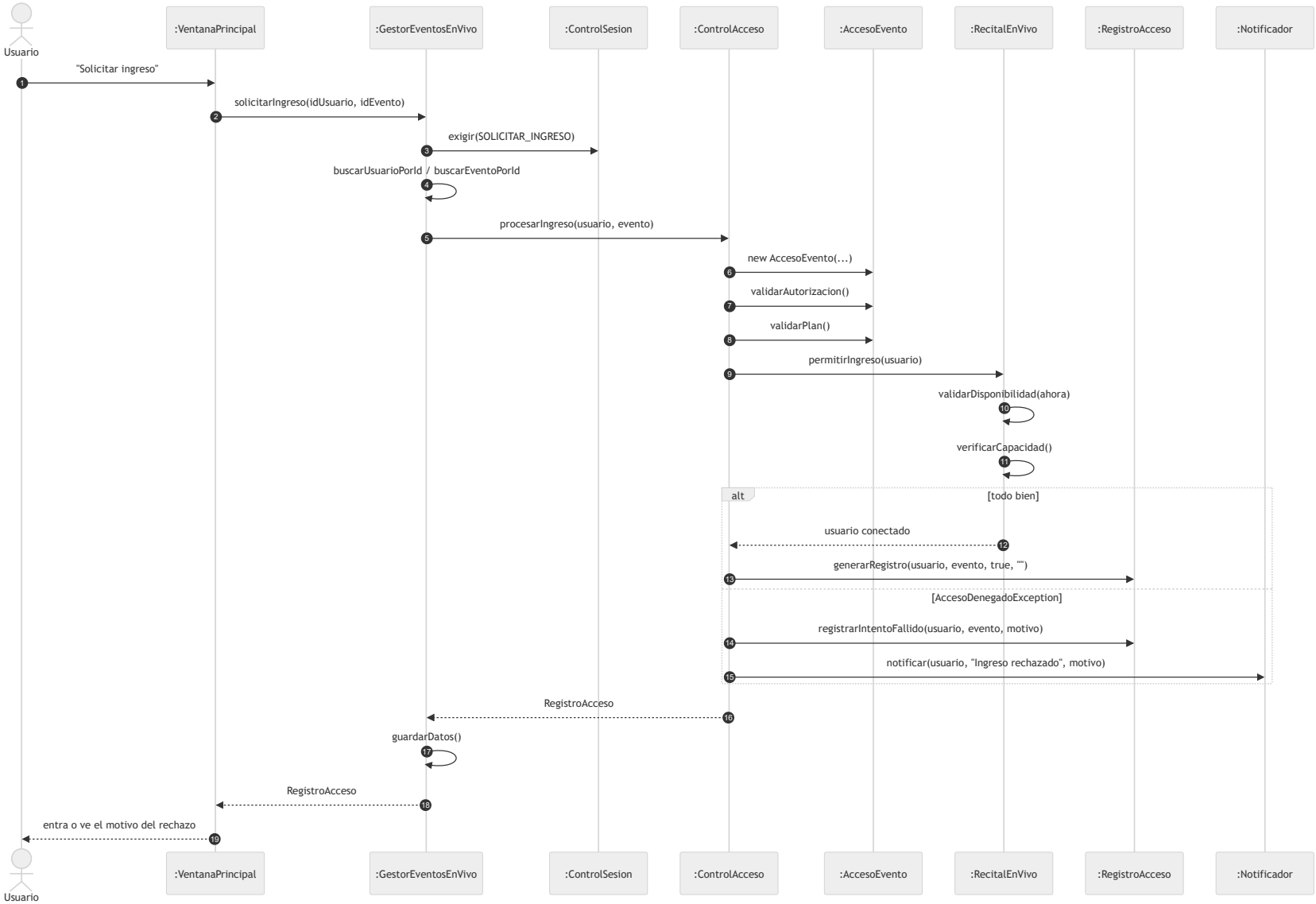
¿Qué objeto recibe los pedidos del usuario y coordina el caso de uso?

`GestorEventosEnVivo` es el controlador. A todos los pedidos les hace lo mismo: verifica el permiso, aplica la operación sobre el modelo y guarda. Los pedidos llegan por tres lados (la ventana, la consola y la API del servidor) y los tres terminan en el mismo controlador. La ventana y la consola llegan por `ServicioEventos`; la API (`ManejadorApi`) llama directamente al gestor del servidor. De esta forma la lógica no queda repartida en las pantallas.


«vista»
VentanaPrincipal
pide
«vista»
MenuAcciones
pide
«interface»
ServicioEventos
«controlador»
GestorEventosEnVivo
implementa
+solicitarIngreso(idUsuario: int, idEvento: int) : RegistroAcceso
+crearEvento(datos: DatosRecital, artista: Artista) : RecitalEnVivo
+iniciarEvento(idEvento: int) : void
+pausarEvento(idEvento: int) : void
+reanudarEvento(idEvento: int) : void
+finalizarEvento(idEvento: int) : void
+cancelarEvento(idEvento: int) : void
+expulsarUsuario(idEvento: int, idUsuario: int) : Usuario
+alternarActivo(idUsuario: int) : Usuario
«servidor»
ManejadorApi
-gestor: GestorEventosEnVivo
pide
aplica las reglas en
«paquete modelo»
Modelo
Evento
RecitalEnVivo
Usuario
Artista

Secuencia completa: solicitar ingreso

Este diagrama junta los cinco patrones en el caso de uso principal. El controlador recibe el pedido y verifica el permiso. Después delega en `ControlAcceso`, que crea el `AccesoEvento` (creador). Los que validan son los expertos: `AccesoEvento` y `RecitalEnVivo`. Al final queda el registro y se guarda.


Usuario
:VentanaPrincipal
:GestorEventosEnVivo
:ControlSesion
:ControlAcceso
:AccesoEvento
:RecitalEnVivo
:RegistroAcceso
:Notificador
"Solicitar ingreso"
solicitarIngreso(idUsuario, idEvento)
exigir(SOLICITAR_INGRESO)
buscarUsuarioPorId / buscarEventoPorId
procesarIngreso(usuario, evento)
new AccesoEvento(...)
validarAutorizacion()
validarPlan()
permitirIngreso(usuario)
validarDisponibilidad(ahora)
verificarCapacidad()
alt
[todo bien]
usuario conectado
generarRegistro(usuario, evento, true, "")
[AccesoDenegadoException]
registrarIntentoFallido(usuario, evento, motivo)
notificar(usuario, "Ingreso rechazado", motivo)
RegistroAcceso
guardarDatos()
RegistroAcceso
entra o ve el motivo del rechazo
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19